Week 5 — Level 2: Intermediate

Dictionaries and Secret Codes

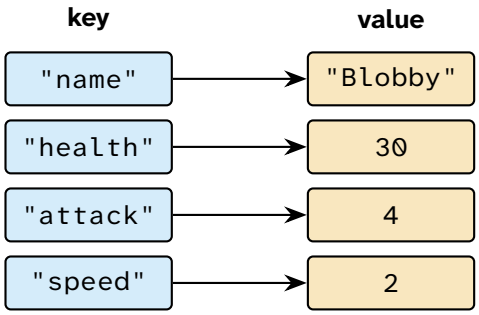
A dictionary looks things up by name, not by number, and can even hold a secret code.

What you will learn

- How a **dictionary** stores values under names called **keys**.
- How to change values, add new keys and loop over everything with `.items()`.
- How to encode and decode secret messages with a **Caesar cipher**.
- What a `KeyError` means, and why real passwords must stay secret.

The big idea: look things up by name

A list finds things by number: `backpack[0]`. A dictionary finds things by name. Each name is a **key**, and the thing it gives back is its **value**. Dictionaries use curly brackets, with a colon between key and value.



`monster["health"]` gives back `30`

A monster as a dictionary

```
monster = {"name": "Blobby", "health": 30, "attack": 4, "speed": 2}
print(monster["health"])          # 30
monster["health"] = 40            # change a value
monster["color"] = "green"        # add a new key
for key, value in monster.items():
    print(f"{key}: {value}")      # every key with its value
```

Bug hunt: KeyError

If you look up a key that is not there, Python stops with a `KeyError`:

Output

```
KeyError: 'magic'
```

The last line shows the key Python could not find. Check the spelling: `"Health"` and `"health"` are different keys! To be safe, check first: `if "magic" in monster:`

Secret codes: the Caesar cipher

Julius Caesar sent secret messages by moving every letter a few places along the alphabet. With a **shift of 3**:

plain: a b c d e f g h i j k l m
secret: **d e f g h i j k l m n o p**

plain: n o p q r s t u v w x y z
secret: **q r s t u v w x y z a b c**

So `hi` becomes `kl`. At the end the alphabet wraps round: x, y, z become a, b, c. To **decode**, shift the other way. That table is a lookup, so in Python it is a dictionary: `{"a": "d", "b": "e", ...}`.

Tip: Real secrets need real protection

A Caesar cipher is easy to crack: there are only 25 shifts to try! Real apps use much stronger **encryption**. Your real passwords stay between you and your parent or carer. Never type a real password into a program you or a friend wrote.

Exercise 5.1: Monster Stats

Folder: `exercises/week05/ex-monster-stats`. Open `starter.py` and press **Run**.

1. TODO 1: in `stat_report()`, loop with `for key, value in monster.items():` and build lines like `health: 30`.
2. TODO 2: `level_up()` adds 10 to health and 2 to attack.
3. Run it and type `magic`. Write the last line of the error:

   _____

4. TODO 3: fix `get_stat()` so a missing stat gives `0`. Then check your work with `check_my_`

`work.py`.

Exercise 5.2: Caesar Cipher

Folder: `exercises/week05/ex-caesar-cipher`. Run `starter.py` first and look at the dictionary it prints.

1. TODO 1: finish `encode()`. For each letter: if it is in the key, add `key[letter]`; otherwise add the letter itself.
2. TODO 2: finish `decode()`. Hint: shifting by `-shift` goes backwards.
3. Encode a kind message for your partner. Secret version: _____ shift: _____
4. Decode this one (shift 3) on paper or with your program: `zhoo grqh!` _____
5. Check your work with `check_my_work.py`.

Vocabulary box

dictionary A collection that stores values under names (keys), written with curly brackets.

key The name you look something up by, like `"health"`.

value What a key gives back, like `30`.

.items() Gives every key together with its value, so a loop can use both.

KeyError An error that means “there is no such key”. Check the spelling!

cipher A way of turning a message into a secret code.

encode / decode Turn a message into secret code / turn it back again.

encryption Strong secret codes that computers use to keep data safe.

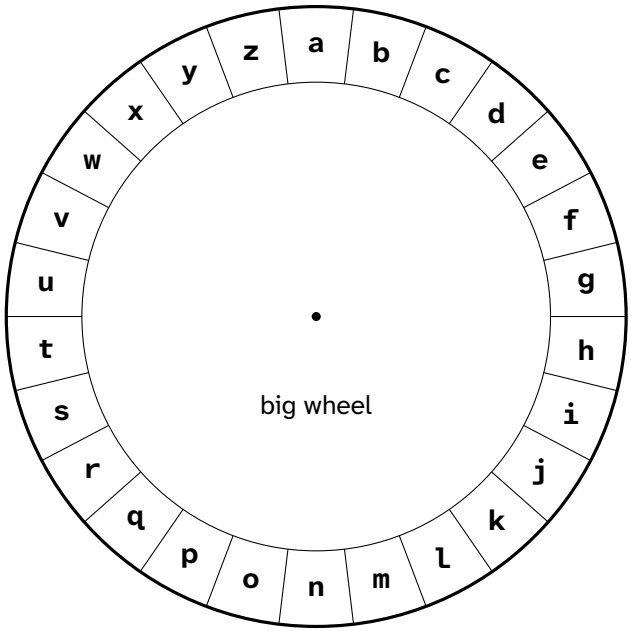
Reflection

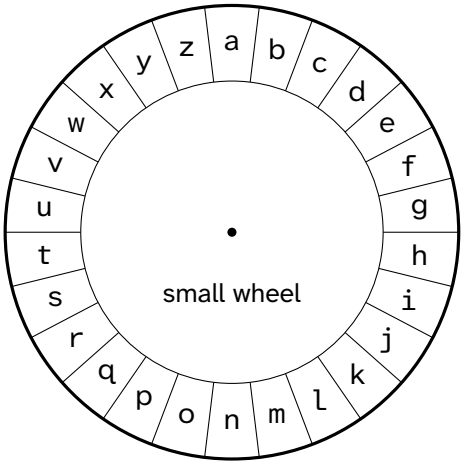
Give a game example where a dictionary is better than a list:

Why should a real password never go into a program or a message?

Cut-out: your Caesar cipher wheel

Cut out both wheels along the thick outer circles. Put the small wheel on top of the big one and push a split pin (or a pencil point) through both centers. Turn the small wheel so that small **a** sits under big **d**: that is shift 3. Encode: small wheel → big wheel. Decode: big wheel → small wheel.





Week 5

Dictionaries and Secret Codes

Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Keys and values (paper, 3 minutes)

```
pet = {"name": "Mochi", "legs": 4, "can_fly": False}
```

1. What does `pet["legs"]` give back? _____
2. Write a line that adds the key `"color"` with the value `"purple"`:

   _____
3. What happens with `pet["Legs"]`? _____

★ Core (everyone): Word Counter (Thonny, 10 minutes)

Open `exercises/week05/hw-word-counter/starter.py`. It should count how many times each word appears, using a dictionary.

1. Press **Run**. Every word says 1. That is a bug!
2. TODO 1: if the word is already in `counts`, add 1 to its count. Otherwise start it at 1.
3. TODO 2: finish `most_common()` with `for word, count in counts.items():`.
4. Run it. Which word is the most common? _____
5. Open `check_my_work.py` and press **Run**.

Two new string tools are used: `text.lower()` makes all letters small, and `text.split()` cuts the text into a list of words.

★★ Challenge (want more?): Crack the code

This message was made with a Caesar cipher, but nobody told you the shift. Use your paper wheel (or your Caesar program from class) to try shifts until it makes sense. Tip: try the short word first.

`lw lv vdih wr sodb`

The shift was _____. The message is: _____

Creative (make it yours): Invent your own code

Make up your own secret alphabet: draw a symbol for every letter (a star for a, a wave for b ...). Write it as a key table, then write a short message for someone at home and see if they can decode it with your key.

Self-check: tick each box when it is true

- ☐ I can explain what a key and a value are.
- ☐ I know what a `KeyError` means and how to avoid it.
- ☐ My word counter counts repeated words correctly.
- ☐ I know that real passwords stay private and never go into a program.